Coleções

Orientação a Objetos – DCC025

2012.3

Coleção

- Definição
  - Uma coleção é uma estrutura de dados que permite armazenar vários objetos
  - Incorpora-se a funcionalidade de
    - Criar
    - Destruir
    - Localizar e
    - Organizar (ordenar, filtrar) as instâncias agregadas à coleção

Tipos de Coleções

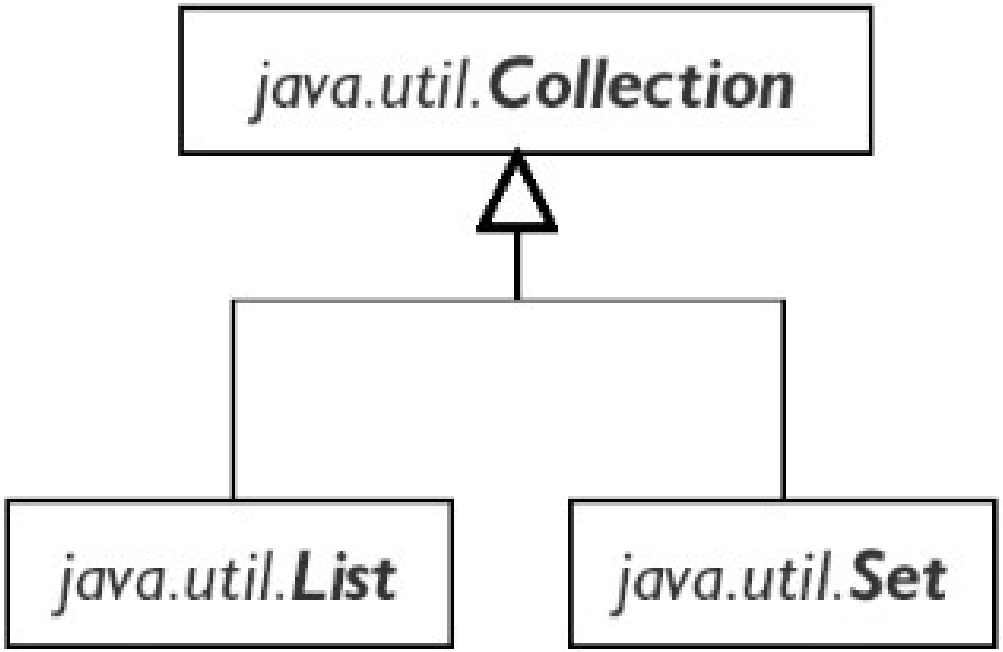
- Tipos
  - Dependendo da forma de fazer 4 operações básicas (adição, remoção, acesso e pesquisa), vários tipos de coleções são definidas. Certas operações poderão ter um desempenho melhor ou pior ou poderão ter restrições ou funcionalidade especial dependendo do tipo de coleção

Tipos de Coleções

- Tipos
  - Os 3 grandes tipos de coleções
    - Listas
    - Conjuntos
    - Mapas

Framework para Coleções

Coleções de elementos individuais



- seqüência definida
- elementos indexados

- seqüência arbitrária
- elementos não repetem

Coleções de pares de elementos

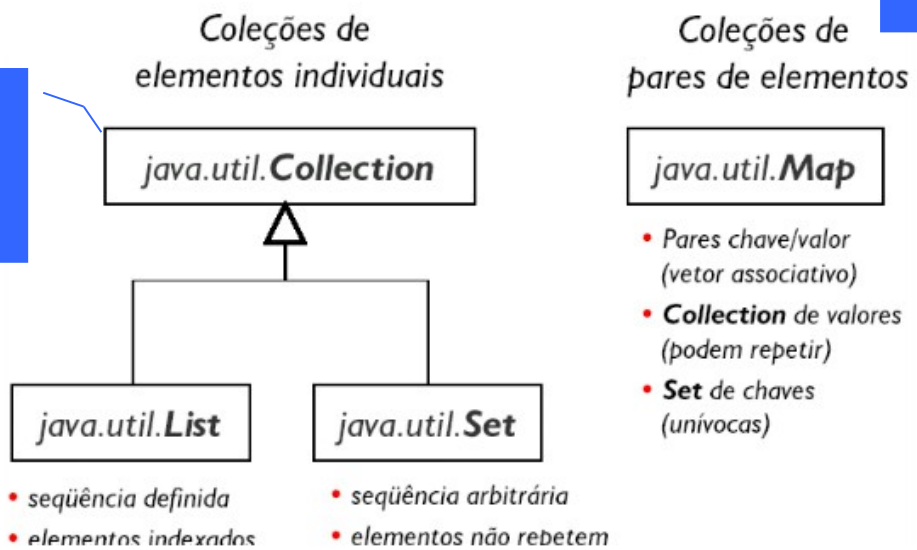
java.util.**Map**

- Pares chave/valor (vetor associativo)
- **Collection** de valores (podem repetir)
- **Set** de chaves (unívocas)

Framework para Coleções

Idéia de chave/valor
Associa pares de objetos

Define operações comuns a todas as subclasses

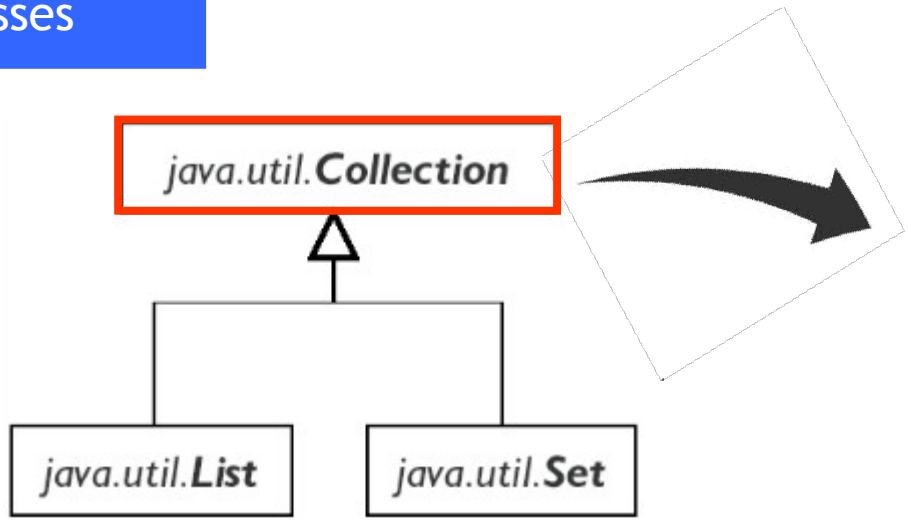


Pode conter elementos repetidos

Não pode conter elementos repetidos

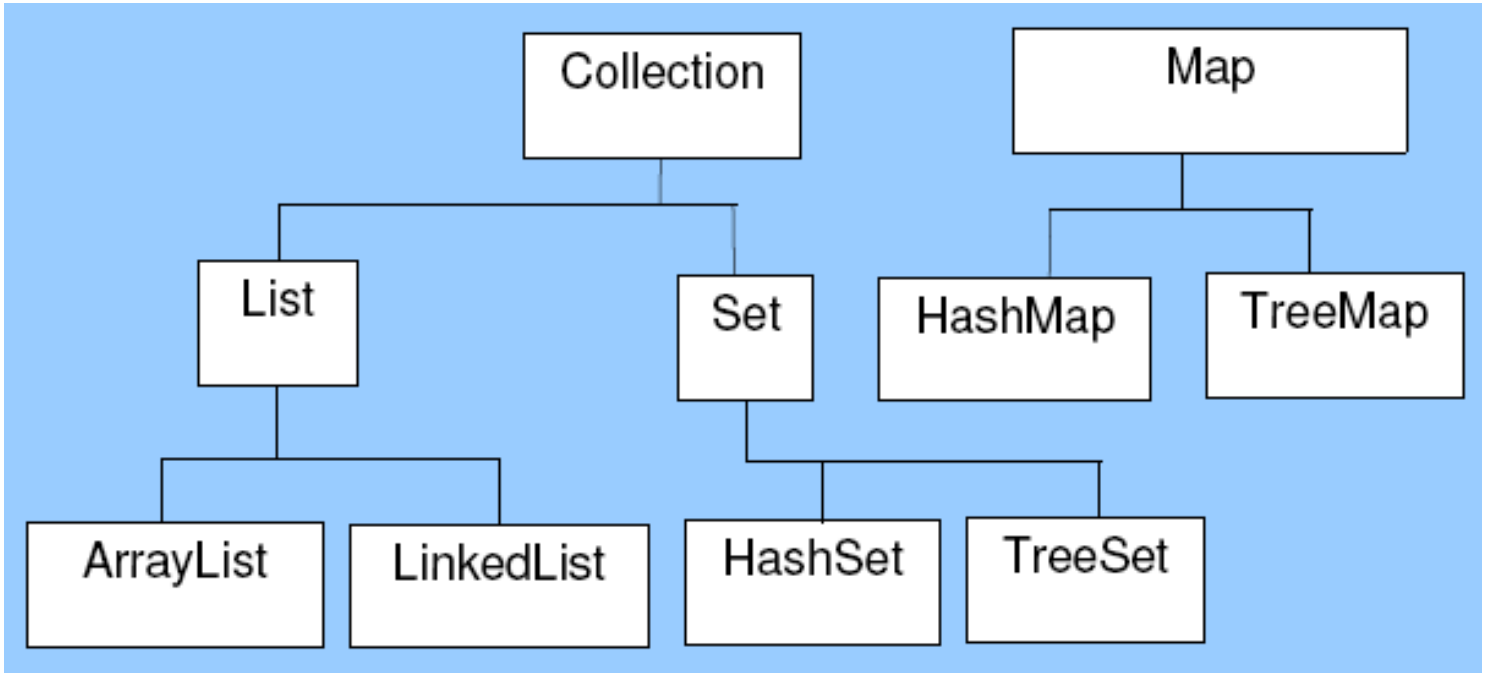
Framework para Coleções

Define operações comuns a todas as subclasses




```
size()
isEmpty()
clear()
add(Object)
contains(Object)
remove(Object)
containsAll(Collection)
removeAll(Collection)
retainAll(Collection)
toArray()
```

Coleções


Collection
Map
List
Set
HashMap
TreeMap
ArrayList
LinkedList
HashSet
TreeSet

Listas

Listas

- Definição
  - Coleção de elementos arrumados numa ordem linear
  - Cada elemento tem um antecessor (exceto o primeiro)
  - Cada elemento tem um sucessor (exceto o último)

Listas

- Características
  - Normalmente implementada como "Array" ou "Lista Encadeada"
  - A Lista pode ser mantida ordenada ou não

Interface List

- List
  - Pode possuir valores repetidos
  - Principais subclasses
    - Vector
    - ArrayList
    - LinkedList

Exemplo List

```
public class Principal {

    public static void main(String args[]){

        ArrayList<Funcionario> funcionarios1 = new ArrayList<Funcionario>();
        LinkedList<Funcionario> funcionarios2 = new LinkedList<Funcionario>();
        Vector<Funcionario> funcionarios3 = new Vector<Funcionario>();

    }

}
```

Exemplo List

```
public static void main(String[] args) {
    ArrayList<String> nomes = new ArrayList<String>();
    nomes.add("João");
    nomes.add("José");
    nomes.add("Maria");
    nomes.add("Bianca");
    System.out.println("Qtd elementos: "+nomes.size());
    String string = nomes.get(3);
    System.out.println(string);
}
```

Exemplo List

```
public class Principal {

    public static void main(String args[]){

        ArrayList<Funcionario> funcionarios = new ArrayList<Funcionario>();

        funcionarios.
    }

}
```

- add(Funcionario e) : boolean - ArrayList
- add(int index, Funcionario element) : void - ArrayList
- addAll(Collection<? extends Funcionario> c) : boolean - ArrayList
- addAll(int index, Collection<? extends Funcionario> c) : boolean - A
- clear() : void - ArrayList
- clone() : Object - ArrayList
- contains(Object o) : boolean - ArrayList
- containsAll(Collection<?> c) : boolean - AbstractCollection
- ensureCapacity(int minCapacity) : void - ArrayList
- equals(Object o) : boolean - AbstractList
- get(int index) : Funcionario - ArrayList

oblems | @ Javadoc | Declaratic

Conjunto

Interface Set

- Set
  - Interface que define uma coleção, ou conjunto, que não contém duplicatas de objetos.
    - Isto é, são ignoradas as adições caso o objeto ou um objeto equivalente já exista na coleção. Por objetos equivalentes, entenda-se objetos que tenham o mesmo código

Interface Set

- Set
  - Principais subclasses
    - HashSet
    - TreeSet
    - LinkedHashSet

Exemplo Set

```
public static void main(String[] args) {
    Set<String> nomes = new HashSet<String>();
    nomes.add("Joao");
    nomes.add("Jose");
    nomes.add("Maria");
    nomes.add("Bianca");
    System.out.println("Qtd elementos: " + nomes.size());
    if (nomes.contains("Maria"))
        System.out.println("Contém Maria");
}
```

Mapas

Mapas

- Definição
  - Um mapa armazena pares (chave, valor) chamados itens
  - Chaves e valores podem ser de qualquer tipo
  - A chave é utilizada para achar um elemento rapidamente
  - Mapas são estruturas especiais usadas para que a pesquisa seja rápida
  - Diz-se, portanto, que um mapa "mapeia chaves para valores“

Mapas

- Características
  - O Mapa pode ser mantido ordenado ou não (com respeito às chaves)
  - Normalmente implementada como "Tabela Hash" ou "Árvore"

Interface Map

- Map
  - Trabalham com o conceito de chave/valor
  - Chaves são únicas (valores podem ser iguais, para chaves diferentes)
  - Principais subclasses
    - LinkedHashMap
    - HashMap
    - TreeMap

Interface Map

- Map
  - Principais métodos
    - **put**(chave,valor)
      Coloca par chave/elemento no mapa
    - **get**(chave)
      Retorna o valor corespondente a chave passada como parâmetro

Exemplo Map

```
public static void main(String[] args) {
    HashMap<String,String> nomes = new HashMap<String,String>();
    nomes.put("joao", "Joao Da Silva");
    nomes.put("jose", "Jose Matos");
    nomes.put("maria", "Maria das Dores");
    nomes.put("bianca","Bianca Patrícia");
    System.out.println("Qtd elementos: "+nomes.size());
    String nome = nomes.get("maria");
    System.out.println(nome);
}
```

Percorrer Coleções

Percorrendo Coleções

- Duas formas de percorrer Coleções
  - Iterator
  - For-each

Percorrendo Coleções

- Interface Iterator
  - Objeto que permite que todos os elementos da coleção sejam acessados
  - Define métodos para percorrer Collections:

Percorrendo Coleções

- iterator()
  - Método que deve retornar um objeto Iterator para a coleção.
  - A coleção deve implementar esse método
  - Uma vez implementado, os seguinte métodos podem ser utilizados:

Percorrendo Coleções

- Interface Iterator
  - hasNext()

    Devolve o valor true se ainda existir objeto a ser percorrido na coleção.

    Indica se chegamos ou não ao fim da coleção

  - next()

    Devolve o próximo objeto da coleção.

Percorrendo Coleções

- Interface Iterator
  - Percorre a coleção sequenciamento, do início ao fim

```java
3  public class Pessoa {
4
5      private String nome;
6
7      public Pessoa(String nome){
8          this.nome = nome;
9      }
10     public String getNome() {
11         return nome;
12     }
13
14     public static void main(String args[]){
15
16         Pessoa p1 = new Pessoa("Pessoa1");
17         Pessoa p2 = new Pessoa("Pessoa2");
18         Pessoa p3 = new Pessoa("Pessoa3");
19
20         ArrayList<Pessoa> conjuntoPessoas = new ArrayList<Pessoa>();
21
22         conjuntoPessoas.add(p1);
23         conjuntoPessoas.add(p2);
24         conjuntoPessoas.add(p3);
25
26         Iterator<Pessoa> it = conjuntoPessoas.iterator();
27
28         while(it.hasNext()){
29             System.out.println(it.next().getNome());
30         }
31     }
32 }
```

Usando for each:

```
4  public class Pessoa {
5
6      private String nome;
7
8      public Pessoa(String nome){
9          this.nome = nome;
10     }
11     public String getNome() {
12         return nome;
13     }
14
15     public static void main(String args[]){
16
17         Pessoa p1 = new Pessoa("Pessoa1");
18         Pessoa p2 = new Pessoa("Pessoa2");
19         Pessoa p3 = new Pessoa("Pessoa3");
20
21         ArrayList<Pessoa> conjuntoPessoas = new ArrayList<Pessoa>();
22
23         conjuntoPessoas.add(p1);
24         conjuntoPessoas.add(p2);
25         conjuntoPessoas.add(p3);
26
27         for(Pessoa p: conjuntoPessoas){
28             System.out.println(p.getNome());
29         }
30     }
31 }
```

Exercícios

Exercício 01

- Teste de Coleções
  - Escreva um programa Java que leia Strings da entrada padrão, armazenando-as em uma coleção, até encontrar o String fim, depois leia novamente vários Strings da entrada padrão até encontrar a palavra fim e diga se cada uma delas foi lida anteriormente ou não anteriormente.

Entrada

Antonio
Matias
Rodrigo
fim
Antonio
Jeronimo
Carlos
Alexandre
fim

Saída:

Antonio localizado
Jeronimo nao localizado
Carlos nao localizado
Alexandre localizado

Exercício 2

- Uso de Coleções
  - Uma loja vende CDs e DVDs. Um usuário, ao entrar na loja, escolhe os produtos de interesse (que podem ser CD ou DVD) para comprar. Desenvolva uma aplicação capaz de calcular o preço final da compra do usuário. Obrigatório o uso de coleções. Considere criar uma classe Venda, que recebe um conjunto de produtos e, para cada produto, informe seu título, código e preço. A classe Venda deve calcular o valor total da venda e informar isso ao usuário.

Exercício 03

- Coleções
  - Crie uma classe Teste com um atributo coleção do tipo String. Crie métodos para adicionar strings na coleção, imprimir a coleção e imprimir a coleção ordenada. Considere como string nome de funcionários. Use a Interface ArrayList.

Exercício 4

- Implemente uma classe chamada “Locadora de DVD” e uma outra chamada “Locadora de CD”. Considere que ambas contêm os seguintes atributos: nome, endereço e telefone da “Locadora”. Além disso, elas implementam o código do produto, gerado automaticamente no momento do cadastro. Cada produto possui um código, diferente dos demais, implementado utilizando uma variável de classe. Cada locadora pode possuir, no máximo, 100 produtos em estoque. As classes “Locadora de DVD” e “Locadora de CD” ainda possuem, como atributo, a quantidade de produtos em estoque. O valor da multa para cada dia de atraso de um produto é de R$2,00. Escreva um programa para Cadastrar produtos nas locadoras;

Exercício 05

- Problema em Coleção
  - Modifique o exercício 04, criando uma Classe funcionário, com um atributo nome, métodos get e set e o construtor que inicialize o atributo. Na classe Principal, crie 3 instância de Funcionário (Ana, Bianca e Carolina). Faça as alterações necessárias (ou implemente novamente, preferencialmente) para que esses 3 objetos sejam inseridos em um conjunto na classe Teste (use a interface ArrayList). Imprima os nomes dos funcionários em ordem alfabética. Crie um método também para imprimir os nomes no sequência em que foram adicionados na lista.